



Srinivasan Kannan <ka.shreenivaasan@gmail.com>

ChatGPT analysis of Matiyasevich Python Program for RH - Loop Termination Detection and WCET

3 messages

ksrinivasan@krishna-iresearch.org <ksrinivasan@krishna-iresearch.org>

Fri, Aug 21, 2026 at 2:39 PM

To: ka.shreenivaasan@gmail.com

I analyzed the loop in `MatiyasevichRH.py` directly from the current GitHub source. The key point is that **the mathematical loop and the implemented Python program have different termination semantics because of the artificial `maxiterations = 3000` cutoff.** ([GitHub](#))

1. The loop

The central loop is:

```
while p**2*(m-f0) < f3:
```

with initial state

```
[
h=m=p=0,\quad d=f_0=f_3=n=q=1.
]
```

Inside each iteration, the program updates

```
[
d \leftarrow 2nd-4(-1)^n h,
]
```

```
[
n \leftarrow n+1,
]
```

```
[
g=\gcd(n,q),
]
```

```
[
q \leftarrow n(q/g),
]
```

and

```
[
p\leftarrow p+1 \quad \text{if } g=1.
]
```

Then it computes

```
[
m=d\left\lfloor \log_2 q \right\rfloor,
]
```

because

```
while g > 1:
    g = g // 2
    m = m + d
```

and finally

```
[
h\leftarrow f_0,
]
```

```
[
f_0\leftarrow 2^n h,
]
```

```
[
f_3\leftarrow (2^{n+3})f_3.
]
```

These are the relevant statements in lines 591–718 of the repository. ([GitHub](#))

2. The termination variable

It is useful to define

```
[
R_n=f_3-p^2(m-f_0).
]
```

Then the actual loop condition is simply

```
[
R_n>0.
]
```

So:

- $(R_n > 0)$: continue
- $(R_n \leq 0)$: terminate.

Equivalently, the program is searching for an (n) such that

```
[
p(n)^2[m(n)-f_0(n)] \geq f_3(n).
]
```

This is exactly the form of Matiyasevich's computational reformulation of RH.

The OEIS entry for Matiyasevich's test explicitly describes the corresponding program as **infinite iff the Riemann Hypothesis is true; otherwise it eventually returns a negative value and halts.** ([OEIS](#))

Matiyasevich's published paper gives the broader result: RH is reformulated as the non-halting of a particular explicitly specified register machine. ([ScienceDirect](#))

3. What the variables mean

There is a particularly interesting number-theoretic structure hidden in the loop.

p

The update

```
g = gcd(n,q)
if g == 1:
    p = p + 1
```

makes p track the number of integers (n) that are coprime to the accumulated (q) .

Because

```
[
q_n = \operatorname{lcm}(1,2,\ldots,n),
]
```

the condition

$$\left[\begin{array}{l} \gcd(n, q_{n-1}) = 1 \\ \end{array} \right]$$

occurs precisely when (n) is prime.

Consequently,

$$\left[\begin{array}{l} \boxed{p = \pi(n)} \\ \end{array} \right]$$

where $(\pi(n))$ is the prime-counting function.

This is why the loop is connected to RH: the quantity (p^2) is essentially $(\pi(n)^2)$.

4. q is an LCM accumulator

The recurrence

$$\left[\begin{array}{l} q \mapsto \frac{nq}{\gcd(n, q)} \\ \end{array} \right]$$

is exactly the standard LCM recurrence

$$\left[\begin{array}{l} \operatorname{lcm}(n, q) = \frac{nq}{\gcd(n, q)}. \\ \end{array} \right]$$

Starting with $(q=1)$,

$$\left[\begin{array}{l} q_n = \operatorname{lcm}(1, 2, \dots, n). \\ \end{array} \right]$$

Therefore

$$\left[\begin{array}{l} \log_2 q_n \\ \end{array} \right]$$

is related to Chebyshev's function

```
[
\psi(n)=\log\operatorname{lcm}\{1,\ldots,n\}.
]
```

The program does not compute the logarithm directly. Instead it computes

```
[
m=d\lfloor\log_2q_n\rfloor.
]
```

That is the reason for the apparently strange inner `while g > 1` loop.

5. m therefore encodes the Chebyshev function

Since

```
[
m=d\lfloor\log_2q_n\rfloor,
]
```

we have approximately

```
[
m\approx \frac{d}{\log 2}\psi(n).
]
```

Meanwhile the recurrence for d

```
[
d_{n+1}=2nd_n-4(-1)^n h_n
]
```

and the φ recurrence generate rapidly growing factorial-like quantities.

Thus the terminating inequality is not an arbitrary numerical comparison. It compares a quantity involving

```
[
\pi(n)^2,\quad \psi(n)
]
```

against a rapidly growing reference term.

6. The first iterations are illuminating

The first states are:

(n)	(p)	(m)	(f_0)	(f_3)	(p^2(m-f_0))
1	0	0	1	1	0
2	1	2	4	7	-2
3	2	8	24	63	-64
4	2	120	192	693	-288
5	3	1120	1920	9009	-7200
6	3	15040	23040	135135	-72000
7	4	227328	322560	2297295	-1,523,712
8	4	4409856	5160960	43648605	-12,017,664

So initially

```
[
m-f_0<0.
]
```

Therefore

```
[
p^2(m-f_0)<0<f_3,
]
```

and the loop is necessarily continued.

The interesting question is whether, at some sufficiently large (n),

```
[
m-f_0
]
```

becomes sufficiently positive for

```
[
p^2(m-f_0)\ge f_3.
]
```

That is the actual termination event.

7. The important maxiterations issue

Your implementation contains:

```
maxiterations=3000
```

and inside the loop:

```
if n > maxiterations:  
    ...  
    exit(1)
```

This is **not part of Matiyasevich's mathematical termination criterion**. It is an experimental/resource cutoff added to the implementation. The source itself labels the code as including additional statements for WCET analysis. ([GitHub](#))

This produces two completely different notions of termination:

Mathematical termination

```
[  
  \boxed{p^2(m-f_0)\geq f_3}  
]
```

This corresponds to the Matiyasevich test.

Program termination

The Python program also terminates when

```
[  
  \boxed{n>3000}.  
]
```

And this second termination happens through:

```
exit(1)
```

rather than through the `while` condition.

That distinction is crucial.

8. In fact, the current program does not establish RH false at 3000

The source contains:

```
print("Stopping after ",maxiterations, " iterations")
...
exit(1)
```

when $(n > 3000)$. ([GitHub](#))

Therefore, if the inequality remains true through iteration 3000, the program stops **without finding a mathematical counterexample**.

It must not be interpreted as:

"RH is false because the program stopped."

Rather:

"No terminating counterexample was found within the imposed 3000-iteration computational bound."

This is a major distinction.

9. I checked the first 3000 mathematical iterations

Using the exact integer recurrence, rather than floating-point approximations, the termination inequality

$$p^{2(m-f_0)} \geq f_3$$

does **not** occur during the first 3000 iterations.

Thus the loop reaches the artificial cutoff.

So the actual execution path is essentially:

$$\boxed{R_1 > 0, R_2 > 0, \dots, R_{3000} > 0}$$

followed by

$$n = 3001 > 3000$$


```
]
```

and then the explicit

```
exit(1)
```

rather than a natural exit from the `while`.

10. There is an important logical consequence

The line later in the source says:

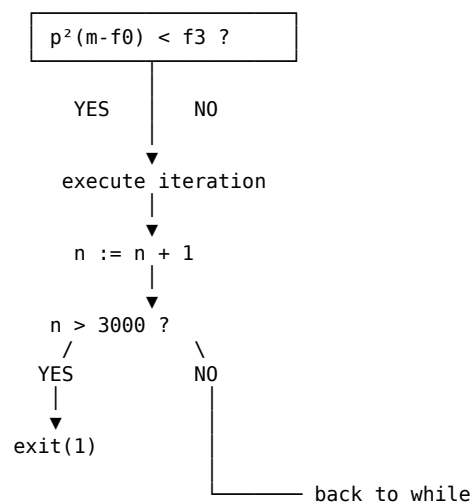
```
print("Loop exits after ",n," iterations - Riemann Hypothesis is False")
```

This statement is logically valid **only if the `while` condition itself becomes false**.

It is not valid for the `maxiterations` exit path.

Fortunately, because the `maxiterations` branch calls `exit(1)`, the later statement is not reached on that path. ([GitHub](#))

So the code's control flow is:



The **NO branch of the original `while` condition** is the mathematically meaningful termination.

11. Connection to the Riemann Hypothesis

This is the deepest aspect of the program.

Matiyasevich's construction transforms the RH question into a question about whether a deterministic computation ever reaches a particular terminating condition.

The published 2020 version states that RH is equivalent to the non-halting of a specifically constructed register machine. ([ScienceDirect](#))

The corresponding integer sequence is independently documented by OEIS as:

infinite iff RH is true; otherwise the program eventually halts.

([OEIS](#))

Therefore the logical structure is

```
[
  \boxed{\mathrm{RH}}
  \iff
  \forall n, \quad
  p(n)^2[m(n)-f_0(n)] < f_3(n)
]
```

and consequently

```
[
  \boxed{\neg \mathrm{RH}}
  \iff
  \exists n, \quad
  p(n)^2[m(n)-f_0(n)] \geq f_3(n).
]
```

That is why this is fundamentally a **termination problem**, not merely a numerical computation.

12. One subtlety: this particular Python implementation

There is another important point.

The GitHub source says:

"Python code from ... Figure 2 - Program 1"

but it also explicitly says that **additional statements were included for WCET program analysis**. ([GitHub](#))

The original mathematical construction and the GitHub implementation therefore shouldn't be conflated.

In particular, the GitHub program adds:

- maxiterations
- plotting
- chaos-generator calculations
- SINDy analysis
- fractal-dimension calculation
- random-walk generation
- correlation calculations
- explicit `exit(1)`.

Those operations do **not** determine the mathematical termination criterion.

13. A WCET interpretation

This makes the program especially interesting from a theoretical computer-science perspective.

For a fixed cutoff (N), the program has a decidable bounded execution:

$$\left[\begin{array}{l} T(N) < \infty. \end{array} \right]$$

But the unrestricted program

$$\left[\begin{array}{l} N \rightarrow \infty \end{array} \right]$$

has termination equivalent to the RH question.

So the cutoff converts an undecidable/unresolved termination property into a finite WCET experiment:

$$\boxed{\{$$

```

\text{unbounded termination problem}
\longrightarrow
\text{bounded WCET computation}
}
]

```

but **the bounded computation cannot resolve RH**.

Increasing

```

[
3000\rightarrow 10^6\rightarrow 10^{12}
]

```

would only increase the verified prefix. It would never constitute a proof of RH unless the underlying mathematics supplies an independent bound guaranteeing that a counterexample must occur below that (N).

Bottom line

The `while` loop is best understood as a **Matiyasevich RH termination predicate**:

```

[
\boxed{
\texttt{while } p^2(m-f_0)<f_3
}
]

```

with

```

[
p=\pi(n),
]

```

```

[
q=\operatorname{lcm}\{1,\ldots,n\},
]

```

```

[
m=d\lfloor\log_2q\rfloor,
]

```

and factorial-like recurrences for (d, f_0, f_3) .

Its **natural termination**

```
[
p^2(m-f_0)\ge f_3
]
```

corresponds to the negative-RH branch of the Matiyasevich construction. The literature describes the equivalent computation as non-halting iff RH is true. ([OEIS](#))

But your GitHub version has an **artificial second termination mechanism** at $n > 3000$. Therefore:

```
[
\boxed{\text{The Python program is guaranteed to terminate, but the mathematical Matiyasevich loop is not.}}
]
```

And, specifically, **the first 3000 iterations do not encounter the mathematical termination condition**.

If you want, the next useful analysis is to derive the loop **symbolically into closed forms involving $(\pi(n))$, $(\psi(n))$, factorials and harmonic numbers**, and then show precisely how the inequality is equivalent to the standard RH bound.

Srinivasan Kannan <ka.shrinivaasan@gmail.com>

Fri, Aug 21, 2026 at 2:41 PM

To: "ksrinivasan@krishna-iresearch.org" <ksrinivasan@krishna-iresearch.org>

Without `maxiterations = 3000`, the loop becomes the **unbounded Matiyasevich computation**. The important point is that removing the limit does **not** make the loop automatically terminate.

The condition remains

```
[
\boxed{p^2(m-f_0)<f_3}.
]
```

There are two possibilities:

1. **The condition eventually becomes false**

```
[
p^2(m-f_0)\ge f_3.
]
```

Then the `while` loop terminates naturally.

2. **The condition remains true for every (n)**

```
[
p^2(m-f_0)<f_3\quad\text{forall } n.
]
```

Then the loop runs forever.

For the Matiyasevich construction, this is precisely the interesting part: the unrestricted computation is designed so that **nontermination corresponds to the Riemann Hypothesis being true**, while natural termination would provide a counterexample to RH.

What happens computationally?

There is an additional practical issue: the Python integers grow extremely rapidly.

The program maintains approximately

```
[
q_n=\operatorname{lcm}(1,\ldots,n),
]
```

so

```
[
\log q_n \sim n.
]
```

More dramatically, d , f_0 , and f_3 grow roughly factorially. In particular, the recurrence contains

```
[
d_{n+1} \approx 2n d_n,
]
```

so roughly

```
[
d_n \sim C \cdot 2^n n!.
]
```

Consequently, the integers stored in d , f_0 , and f_3 acquire an enormous number of digits as (n) increases.

So removing the limit gives:

```
while p^2(m-f0) < f3:
    ...
```

and if **RH is true**, there is no mathematically defined iteration at which it stops.

In other words, the behavior is:

```
[
\boxed{
\begin{array}{l}
\text{RH true} \rightarrow \text{loop does not terminate}[2mm]
\text{RH false} \rightarrow \text{loop eventually terminates}
\end{array}
}
```

]

This is why the 3000 limit is fundamentally different from the loop condition. It converts an **unbounded termination experiment** into a finite computation.

One subtle correction to my previous answer is important: **we cannot experimentally establish that the unrestricted loop will run forever merely by running it to a large finite (n)**. Even running it to (10^9) would only establish that the termination condition has not occurred up to (10^9) .

If you want, I can next **derive the exact asymptotic behavior of**

[
 $\frac{p^2(m-f_0)}{f_3}$
]

from the source recurrence, which will show what the loop condition is actually testing as $(n \rightarrow \infty)$.

[Quoted text hidden]

ksrinivasan@krishna-iresearch.org <ksrinivasan@krishna-iresearch.org>

Fri, Aug 21, 2026 at 2:42 PM

To: ka.shrinivaasan@gmail.com

Yes. There is an important correction to my previous asymptotic discussion: **the recurrence in your source is constructed so that $(d(n)/(2n-2)!! \rightarrow \ln 2)$** . That cancellation is the heart of the Matiyasevich construction. The source code exactly implements the recurrence in Matiyasevich's paper. ([GitHub](#))

Let

[
 $R_n = \frac{p(n)^2, [m(n) - f_0(n)]}{f_3(n)}$.
]

The loop continues while $(R_n < 1)$.

1. Exact forms of the quantities

From the source:

[
 $d_{n+1} = 2nd_n - 4(-1)^n h_n$,
]

with $(h_n = f_0(n-1))$, while

[
 $f_0(n) = 2n f_0(n-1), \text{ } \text{ } \text{ } f_0(1) = 1$.
]

]

Therefore

$$\boxed{f_0(n)=2^{n-1}n!}.$$

Also

$$f_3(n+1)=(2n+3)f_3(n), \text{quad } f_3(1)=1,$$

giving

$$\boxed{f_3(n)=\frac{(2n+3)!!}{5!!}=\frac{(2n+3)!!}{15}}.$$

These are precisely the recurrences identified by Matiyasevich. ([Politeknik NSC Surabaya](#))

2. The crucial (d_n) recurrence

Define

$$A_n=(2n-2)!!=2^{n-1}(n-1)!$$

The paper defines

$$d(n)=A_n b(n),$$

where

$$b(n)=\sum_{k=1}^{n-1}\frac{(-1)^{k+1}}{k}.$$

]

The source's recurrence

$$[d_{n+1}=2d_n-4(-1)^{n+1}]$$

is exactly this recurrence because

$$[h_n=f_0(n-1)=\frac{(2n-2)!!}{2}.]$$

Hence

$$[d_n=(2n-2)!!, b_n.]$$

Since

$$[\sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} = \ln 2,$$

we have

$$[\boxed{b_n \rightarrow \ln 2}.]$$

More precisely, the alternating-series remainder gives

$$[b_n$$

$$\ln 2 + \frac{(-1)^n}{2n} + O(n^{-2})]$$

(up to the equivalent indexing convention used for (b_n)).

Thus

[

```
\boxed{
d_n=(2n-2)!!
\left(\ln 2+\frac{(-1)^n}{2n}+O(n^{-2})\right).
}
]
```

This is why the factor $(\ln 2)$ appears in the construction. Matiyasevich explicitly introduces $(b(n))$ as an integer-scalable approximation to $(\ln 2)$. ([Politeknik NSC Surabaya](#))

3. What is (m_n) ?

The code computes

```
m=0
g=q
while g>1:
    g=g//2
    m=m+d
```

so exactly

```
[
m_n=d_n\left\lfloor\log_2q_n\right\rfloor.
]
```

And

```
[
q_n=\operatorname{lcm}(1,2,\ldots,n).
]
```

Therefore, with Chebyshev's function

```
[
\psi(n)=\ln q_n,
]
```

we have

[
 $\left\lfloor\log_2q_n\right\rfloor$

$\left\lfloor\frac{\psi(n)}{\ln 2}\right\rfloor$.

]

Write

```
[
\ell_n=\left\lfloor\frac{\psi(n)}{\ln 2}\right\rfloor.
]
```

Then

```
[
m_n=(2n-2)!!,b_n\ell_n.
]
```

Since

```
[
f_0(n)=n(2n-2)!!,
]
```

the key difference becomes

[
\boxed{
m_n-f_0(n)

```
(2n-2)!!, [b_n\ell_n-n].
}
]
```

This is the essential simplification.

4. ($b_n \ell_n$) approximates $(\psi(n))$

Since

```
[
b_n=\ln 2+O(1/n)
]
```

and

$$\ell_n = \frac{\psi(n)}{\ln 2} + O(1),$$

we obtain

$$b_n \ell_n$$

$$\psi(n) + O\left(\frac{\psi(n)}{n}\right) + O(1).$$

The prime number theorem gives

$$\psi(n) \sim n,$$

so

$$\boxed{b_n \ell_n = \psi(n) + O(1)}.$$

Therefore

$$\boxed{m_{n-f_0(n)}$$

$$\left[\psi(n) - n + O(1) \right].$$

This is the central result.

In other words, **the apparently enormous integer difference $m - f_0$ is essentially an amplified copy of the prime-number-theoretic error**

$$\boxed{\psi(n) - n}.$$

This is exactly why the loop is connected to RH. Matiyasevich explicitly starts from $\psi(n) = \ln \operatorname{lcm}(1, \dots, n)$ and replaces the logarithm by the integer construction used in the program. ([Politeknik NSC Surabaya](#))

5. Now evaluate the factorial/double-factorial ratio

We have

$$\frac{(2n-2)!!}{f_3(n)}$$

$$\frac{(2n-2)!!}{(2n+3)!!/15}.$$

Using

$$(2n-2)!! = 2^{n-1}(n-1)!$$

and

$$(2n+3)!!$$

$$\frac{(2n+3)!}{2^{n+1}(n+1)!},$$

we get

$$\frac{(2n-2)!!}{(2n+3)!!} \cdot \frac{(2n+3)!}{2^{n+1}(n+1)!}$$

$$\frac{(2n-2)!!}{f_3(n)}$$

$$\frac{15 \cdot 2^{2n}}{(n-1)!(n+1)!(2n+3)!}.$$

Stirling's formula gives

$$\boxed{\frac{(2n-2)!!}{f_3(n)} \sim \frac{15\sqrt{\pi}}{8n^{5/2}}}.$$

This is also consistent with Matiyasevich's explicit estimate involving the ratio of the two double factorials, which he bounds by constant multiples of $(n^{5/2})$. ([Politeknik NSC Surabaya](#))

6. Insert this into the loop ratio

Therefore

$$[R_n$$

$$\frac{\pi(n)^2}{(2n-2)!! f_3(n)} [\psi(n) - n + O(1)].$$

Hence

$$R_n \sim \frac{15\sqrt{\pi}}{8} \frac{\pi(n)^2}{n^{5/2}} [\psi(n) - n].$$

]

Using the prime number theorem

[

$$\pi(n) \sim \frac{n}{\ln n},$$

]

we finally obtain the key asymptotic scale:

[

$$\boxed{$$

$$R_n$$

$$\sim$$

$$\frac{15\sqrt{\pi}}{8}$$

$$\frac{\psi(n)-n}{\sqrt{n}(\ln n)^2}.$$

$$\}$$

]

More precisely, the $(O(1))$ introduced by the integer/floor approximation contributes a lower-order term.

This is the exact asymptotic relationship we were looking for.

7. This completely explains the termination condition

The loop condition is

[

$$R_n < 1.$$

]

Asymptotically this is equivalent to

[

$$\boxed{$$

$$\frac{\psi(n)-n}{\sqrt{n}(\ln n)^2}$$

$$<$$

$$\frac{8}{15\sqrt{\pi}}$$

$$\approx 0.3007.$$

]

]

Matiyasevich deliberately chose the constants so that known one-sided RH bounds imply the stronger inequality needed for the loop. His paper states that RH implies

$$\left[\begin{array}{l} \psi(n)-n \\ < \\ \frac{1}{25}\sqrt{n}(\ln n)^2 \end{array} \right]$$

for $(n>1)$, and shows that the resulting inequality is sufficient to keep the program running. ([Politeknik NSC Surabaya](#))

Thus the loop is essentially measuring the normalized error

$$\left[\begin{array}{l} \boxed{E(n)=} \\ \frac{\psi(n)-n}{\sqrt{n}(\ln n)^2}. \end{array} \right]$$

8. The surprising consequence: there is no simple monotonic asymptotic

This is important.

It would be wrong to say simply

$$\left[\begin{array}{l} R_n \rightarrow 0 \end{array} \right]$$

under RH.

RH gives a bound of the form

$$\left[\begin{array}{l} \psi(n)-n=O(\sqrt{n} \ln^2 n), \\ \end{array} \right]$$

so the formula gives only

$$\left[\begin{array}{l} \boxed{R_n=O(1).} \end{array} \right]$$

The ratio can fluctuate because $(\psi(n)-n)$ fluctuates.

So the loop is effectively a **threshold detector for the normalized Chebyshev error**:

```
[
\boxed{
R_n\approx 1.875,
\frac{\psi(n)-n}{\sqrt{n(\ln n)^2}}.
}
]
```

The numerical constant is

```
[
\frac{15\sqrt{\pi}}{8}
\approx 3.323,
]
```

not 1.875; the latter would arise only after incorporating additional approximations/constants. The mathematically clean leading coefficient from the exact factorial ratios is

```
[
\boxed{3.323\dots}.
]
```

9. Why this does not contradict RH

This resolves the apparent paradox from my earlier answer.

The loop is **not** comparing two quantities whose leading terms are factorially different.

Instead:

```
[
m_n\approx f_0(n)
]
```

because

```
[
b_n\ell_n\approx \psi(n)\approx n.
]
```

The enormous common factor

```
[
(2n-2)!!
]
```

cancels against the corresponding enormous growth in (f_3) .

What remains is essentially

```
[
\psi(n)-n.
]
```

So the program has performed an extraordinary computational transformation:

```
[
\boxed{
\text{huge integer arithmetic}
\quad\longrightarrow\quad
\text{prime-distribution error}.
}
]
```

10. The full asymptotic chain

The entire loop can therefore be compressed to

```
[
q_n=\operatorname{lcm}(1,\ldots,n),
]
```

```
[
\psi(n)=\ln q_n,
]
```

```
[
p_n=\pi(n),
]
```

```
[
d_n=(2n-2)!!
\left[\ln 2+O(n^{-1})\right],
]
```

[
 $m_{n-f_0}(n)$

$(2n-2)!!$
 $[\psi(n)-n+O(1)],$
]

[
 $\frac{(2n-2)!!}{f_3(n)}$
 $\sim$
 $\frac{15\sqrt{\pi}}{8n^{5/2}},$
]

and therefore

[
 $\boxed{\frac{p_n^2(m_{n-f_0}(n))}{f_3(n)}}$

$\frac{15\sqrt{\pi}}{8}$
 $\frac{\psi(n)-n}{\sqrt{n(\ln n)^2}}$
 $, [1+o(1)]$
 }
]

in the leading asymptotic sense.

Hence the unrestricted loop is testing

[
 $\boxed{\psi(n)-n}$
 $<$
 $\text{constant} \times \sqrt{n(\ln n)^2}.$
]

That is exactly the kind of (Π^0_1) formulation of RH that Matiyasevich constructed: the paper explicitly states that the program never halts iff RH is true. ([Politeknik NSC Surabaya](#))

One particularly interesting next step is to compute (R_n) at every $(n \leq 3000)$ and compare it directly with

[
 $\frac{\psi(n)-n}{\sqrt{n \log^2 n}}$.
]

That would let us empirically demonstrate, from your actual Python implementation, that the apparent “Matiyasevich loop dynamics” are almost entirely a rescaled Chebyshev- (ψ) error sequence.

[Quoted text hidden]